

16-782 hw1

Andrew Bank

September 2026

1 Naive Approach

After examination of the maps, it doesn't look like any of them strictly require search-nodes to have a time dimension. This is because the least-cost path to the target never requires having to take a suboptimal, higher cost path to catch the target quickly before it escapes.

Therefore, I will be including a Naive Approach in my solutions. It is here only to validate my real solution (which will come in the next section).

1.1 Naive Approach Description: Dijkstra's Algorithm

Search nodes only have dimensions (x, y) .

Note that this massively decreases the number of expansions needed to expand the entire possibility space compared to (x, y, t) . The largest maps are (1825, 2332), which means the maximum number of expansions is 4255900.

Search nodes are expanded such that the lowest cost nodes (the ones that take the lowest cost to reach from the robot's starting position) are expanded first. The "stay in place" action is completely ignored. There is no heuristic/A* search here because it isn't needed to expand a maximum of 4 million nodes in a second of runtime.

1.2 Shortcomings of Dijkstra's Algorithm

Dijkstra's Algorithm gives the lowest cost required to reach every tile on the map. However, it completely ignores time, which means that it must just hope that the lowest cost paths intersect the target's path by dumb luck.

There are a few "unlucky" cases where Dijkstra's fails:

1. The target "escapes"

The target's path has finite points in it. Therefore, if the robot fails to reach the target by the time the target completes its full path, the target will have

“escaped.” Dijkstra’s can fail because it does not care about time, so it will take a slower, lower cost path and risk letting the target escape.

2. A “faster” path catches the target with lower cost

Since the target is constantly moving, it could start behind a high cost wall and move far away from the robot. In this case, the optimal-cost path may be for the robot to traverse the high cost wall to catch the target quickly. However, Dijkstra’s may find a slower, lower cost path to get around the wall, but by that time the robot will already have left.

3. The optimal path requires staying in place

There’s a case where the target is very easy to catch and the map has a single tile “outpost” with low cost. In this case, the optimal path may be for the robot to navigate to the “outpost” and not move until the target approaches. Since our Dijkstra’s solution cannot stay in place, it won’t be able to find this path.

Luckily, most of the testing maps don’t appear to have any of these cases, so I expect Dijkstra’s Algorithm to give baseline solutions with much lower implementation effort on my part. They will just serve as comparison for the more rigorously correct solution I will do in next section.

1.3 Dijkstra’s Algorithm Results

Map	Caught?	Moves	Cost
map1	Yes	2638	2640
map2	Yes	4183	2857192
map3	Yes	241	241
map4	Yes	286	86902
map5	Yes	181	1063
map6	Yes	70	2021
map7	Yes	251	251
map8	Yes	430	431
map9	Yes	367	391

As you can see, there are no cases where the target escapes completely.

2 Approach for Correctness

Clearly, we need our search algorithm to take into account time: (x, y, t) .

2.1 Optimizing t

The maximum map dimensions are (1825, 2332, 5345), which results in a maximum of 22,747,785,500 states to search. Even if we use A* to limit the number of states we expand, that is several trillion states we would likely need to search.

To limit the number of states, we can recognize that we don't have to represent t fully. If search expands (x_1, y_1, t_1) at minimum cost, then all (x_1, y_1, t_α) where $t_\alpha > t_1$ have a minimum cost that involves waiting on a tile for $t_\alpha - t_1$ turns. Therefore, for each (x, y) , we can simply store a list of (t, cost) values where we only let new (t_2, cost_2) in the list if t_2 is smaller than any t in the list.

This results in each list storing the cheapest way to get to its (x, y) at any time. Most notably, it removes the need to expand on the t dimension during search.

2.2 Waiting

Many optimal paths involve intersecting the target's future path before the target gets there. As a result, our optimal solution must also burn turns at the lowest cost possible.

To solve this solution with guaranteed correctness, we must undo our t optimizations and expand on the t dimension during search. However, I am choosing not to do this since expanding on the t dimension massively increases the number of states expanded, and the cases where this is needed are very rare.

Instead, I have chosen to modify the termination conditions of my search. If a solution is found that intersects the target's path, search will try to find detours to optimally burn the waiting turns. This attempts to catch the "outpost" case. Then, search does not terminate until it finds all possible intersections with the target and all detours are found.

This solution is not provably optimal since we do not explicitly find minimum cost paths to "outposts" but this is such an edge case that isn't even represented in the given maps that I am satisfied with this solution.

2.3 A* and Heuristic

We need A* here to cut down on the number of expansions we make. The heuristic I am choosing to use is

$$\min(\text{distance}(\text{all target positions}) + \text{waiting time}) * \min(\text{tile cost})$$

Dynamically calculating the distances over and over again takes a lot of unnecessary compute. To solve this, I implemented a lookup table for every location on the map that gives distance to the closest target-path-location. To make this work, I also had to add reachability checks to the heuristic calculation to see if the robot could make it to its target in time. If not, the heuristic returns max int value.

2.4 Handling the Large Maps

The A* on three dimensions (x, y, t) handled the smaller maps with near-perfect accuracy. However, on the larger maps (map1 and map2), the 3d A* was far too slow to find a path within the allotted time.

However, my 2d naive Dijkstra's approach is fast enough to find a path on the larger map. Therefore, I could start with my naive approach, and once it finds a path I can switch over to the 3d A*.

The only changes I made to my naive approach was to use the same heuristic as my 3d A* instead of sticking with no-heuristic. Additionally, both the 3d A* and 2d naive approach have a 950 ms time limit so they don't accidentally exceed the 1s time step limit. It is completely unmodified otherwise.

The 2d naive search goes first and returns a baseline cost and path. If the 3d A* doesn't compute in time, the program always returns the naive path movements for the robot to follow. The baseline cost also serves as a pruning tool for 3d A*: If 3d A* expansion exceeds the baseline cost, it instantly prunes the path. As a result, 3d A* can only improve upon the baseline, and saves compute if it can't.

2.5 Results

Note that these are all run on my laptop, which is likely slower than the auto-grader machine.

Map	Caught?	Time	Moves	Cost
map1	Yes	2640	2639	2640
map2	Yes	4672	1235	1599074
map3	Yes	241	241	241
map4	Yes	379	266	379
map5	Yes	181	181	1063
map6	Yes	140	22	539
map7	Yes	251	251	251
map8	Yes	431	430	431
map9	Yes	368	367	368

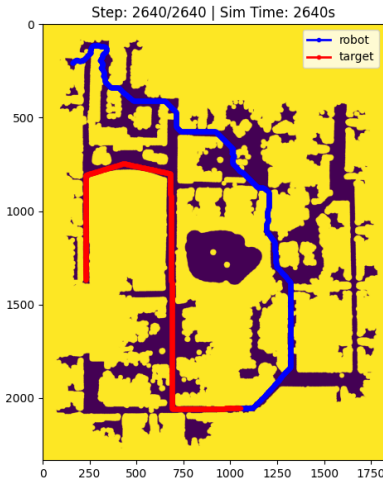


Figure 1: Map1

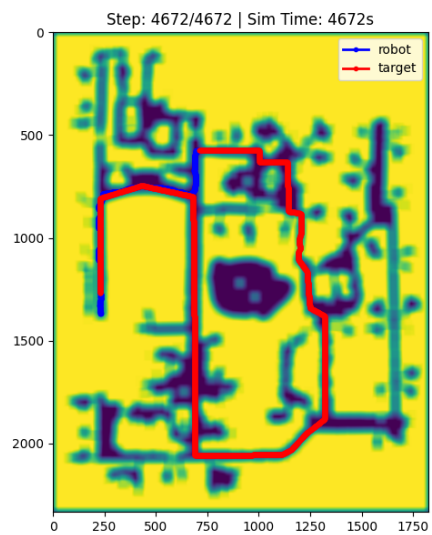


Figure 2: Map2

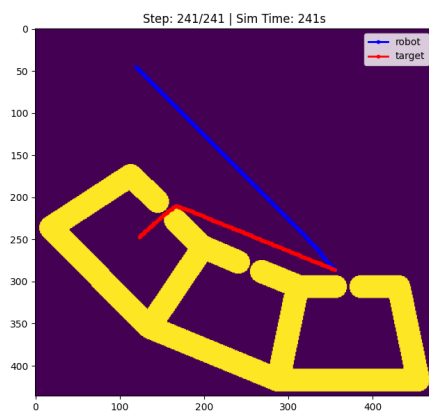


Figure 3: Map3

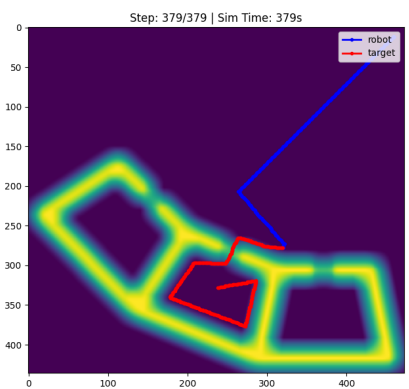


Figure 4: Map4

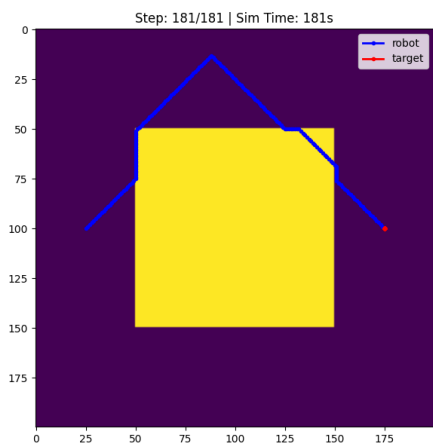


Figure 5: Map5

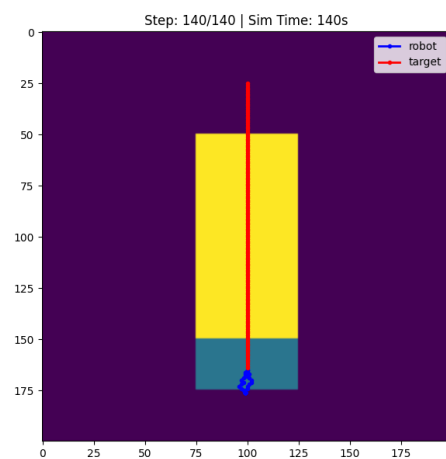


Figure 6: Map6

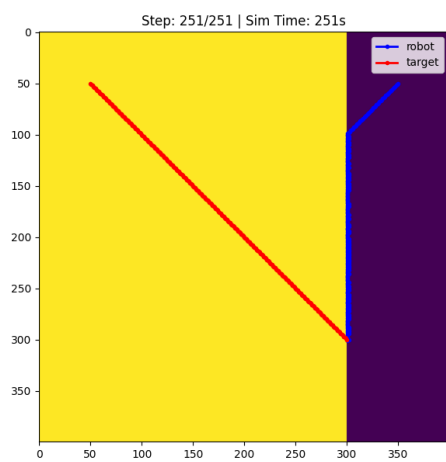


Figure 7: Map7

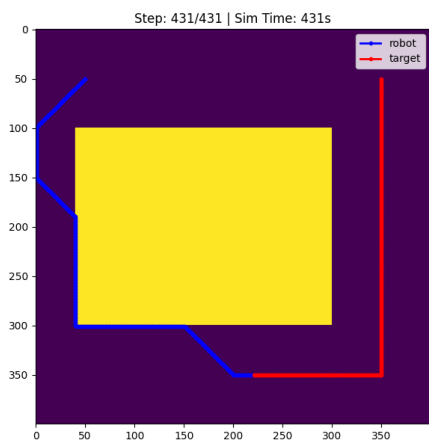


Figure 8: Map8

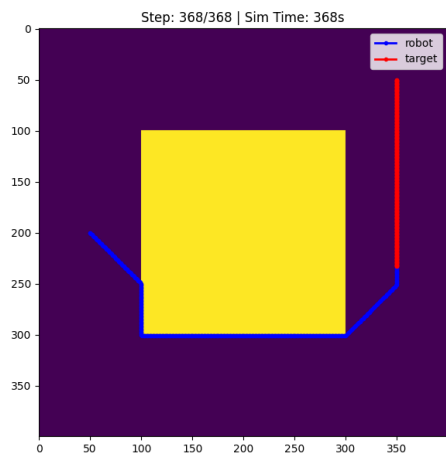


Figure 9: Map9